

Hermes Agent: полный гайд для новичка

Установка, выбор модели,
первый рабочий процесс, цикл обучения

Перевод для @prompt_design

21 апреля 2026 г.

Содержание

Введение	3
1. Что такое Hermes Agent	4
Версия на 30 секунд	4
Версия на 2 минуты	4
Превью: цикл обучения	4
2. Для кого Hermes	5
CLI-кодер	5
Оператор автоматизации	5
Оператор Telegram-бота	5
Какой из них вы?	5
3. Чем Hermes отличается от Claude Code, Cursor и OpenClaw	6
Одной строкой	6
Сравнение	6
Когда что выбирать	6
Скорее всего, вы захотите запустить два	7
4. Установка за 2 минуты	8
Что нужно	8
Установка	8
Первый запуск	8
Проверка	8
Три частые ошибки	8
5. Выбор модели (главная ошибка первых выходных)	10
Ошибка первых выходных	10
Цена и возможности	10
Когда локальная модель подходит, а когда нет	10
Кнопка «просто работай»: Nous Portal	11
6. Ваш первый рабочий процесс	12
Пример 1 – ассистент по коду на вашем ноутбуке	12
Пример 2 – Telegram-бот на VPS за \$5	12
Пример 3 – утренний брифинг по cron	13
7. Цикл обучения	14
Почему Hermes со временем становится лучше	14
Ограниченная память – это фича, а не баг	14
Скиллы – это выполняемые процедуры, а не заметки	14
Правило пяти вызовов инструментов	15
Что на самом деле значит «Harness Engineering»	15
8. Обязательные скиллы для установки в первый день	16
Что такое Hub на 643 скилла	16
Пять, которые стоит поставить сегодня	16
Команды для работы с Hub	16

9. Куда двигаться дальше	17
Путь 1: глубже в агентов	17
Путь 2: глубже в инструментарий	17
Путь 3: глубже в инфраструктуру	17
10. FAQ	18
Hermes Agent бесплатный?	18
Работает ли на Windows?	18
Можно ли запустить полностью офлайн?	18
Нужен ли GPU?	18
Мои данные куда-то отправляются?	18
Hermes vs Claude Code – нужно ли выбирать один?	18
Может ли Hermes сломать мой код?	18
Что, если он что-то сломает?	19
Может ли команда делить одного агента?	19
Можно ли использовать с Ollama или LM Studio?	19
Как обновить?	19

Введение

Автор: Kevin Simback, мейнтейнер Hermes Atlas. Обновлено 20.04.2026. Примерно 18 минут чтения.

Основано на разборе 93 проектов экосистемы, 33 исследовательских источников и актуального GitHub-репозитория. Обновляется ежемесячно.

1. Что такое Hermes Agent

Hermes Agent – это open-source ИИ-агент от Nous Research, который работает на вашей машине или на дешевом VPS, помнит все, что выучил между сессиями, и сам пишет для себя переиспользуемые скиллы прямо в процессе работы. Общается через CLI, Telegram, Discord, email и много других мессенджер-провайдеров. На 20.04.2026 у него 104 791 звезд на GitHub, и это самый быстрорастущий open-source агент 2026 года.

Версия на 30 секунд

Ставите Hermes одной curl-командой. Выбираете провайдера модели (вариантов много) и конкретную модель (Claude, GPT-5.4, GLM-5.1, MiniMax или даже локальную Ollama). Даете ему задачу – «суммаризируй мои уведомления с GitHub каждое утро в 8:00», «помоги отладить этот Python-скрипт» или «следи за этим сайтом и дай мне знать в Telegram, когда цена упадет».

Он работает. Он учится. Через неделю та же задача выполняется точнее – потому что Hermes тихо писал скиллы (маленькие markdown-файлы, в которых зафиксировано, что сработало) и подтягивает их в следующие запуски.

Это весь продукт. Поставить, назначить задачи, дать эффекту накапливаться.

Версия на 2 минуты

Большинство ИИ-инструментов не хранят состояние: открыли чат, задали вопрос, закрыли вкладку – следующая сессия начинается с нуля. Hermes это переворачивает. Он живет на вашем компьютере или на виртуальном сервере. У него есть постоянная память. Он создает скиллы – процедуры, которые можно переиспользовать, – каждый раз, когда что-то понимает. И дотягивается до вас через любой удобный канал: терминал, Telegram-бот, cron-задача, вебхук.

Главная архитектурная ставка – **Harness Engineering** (инженерия обвязки). Nous считает, что настоящий прорыв в 2026 году – не более умная модель, а более умная обвязка вокруг модели. Поэтому Hermes вкладывается в пять слоев вокруг LLM: инструкции, ограничения, обратная связь, память, оркестрация. А саму модель можно менять: сегодня frontier-модель, завтра локальная, послезавтра что-то еще.

Это часто называют «агент, который растет вместе с тобой». Это слоган Nous и заодно полезная ментальная модель: в первый день Hermes – это обобщенный помощник. На тридцатый день это *ваши* помощник с тридцатью днями запомненных предпочтений.

Превью: цикл обучения

Каждые несколько вызовов инструментов Hermes делает паузу и задает себе вопрос: *что только что произошло, что сработало, что не сработало, стоит ли из этого сделать скилл?* Если ответ – да, он пишет файл скилла в `~/.hermes/skills/` и использует его дальше. Скиллы – это обычный markdown. Их можно читать, редактировать или удалять те, которые получились неудачно.

Это единственная функция, которая реально важна, и мы разберем ее в главе 7.

2. Для кого Hermes

Hermes привлекает три типа людей. Поймите, кто из них вы, – и остальная часть гайда встанет на свои места.

CLI-кодер

Вы живете в терминале. Уже используете Claude Code или Cursor для кодинга в редакторе. Хотите агента, который сидит *вне* редактора – чтобы попросить «проведи аудит репо на мертвый код» или «составь план миграции с версии 0.9 на 0.10», не уходя из CLI.

С чего начать: hermes CLI, скиллы, память. **Пока игнорировать:** Telegram, cron, мульти-агентность.

Оператор автоматизации

Возможно, вы даже не пишете код. Вам нужен агент, который делает повторяющиеся задачи – суммирует новости, следит за рынками, готовит отчеты, разбирает инбокс, – пока вы спите. Hermes будет жить на Mac Mini или VPS за \$5, подключенный к нужным каналам.

С чего начать: cron, мессенджер-шлюзы, память. **Пока игнорировать:** выполнение кода, мульти-агентность.

Оператор Telegram-бота

Вам нужен всегда доступный агент, которому можно писать с телефона. Командировка, встречи, спортзал – без разницы, пингуете Hermes, он отвечает. Интеграция с Telegram – киллер-фича.

С чего начать: мессенджер-шлюз, голос, скиллы. **Пока игнорировать:** локальный CLI.

Какой из них вы?

Хотите...	Начните с...	Пока пропустите
Писать лучший код из терминала	CLI + скиллы	Telegram, cron
Автоматизировать повторяющиеся задачи	Cron + шлюз	Выполнение кода
Общаться с агентом с телефона	Telegram + голос	CLI

Потом все три варианта можно совмещать. На первой неделе выберите один.

3. Чем Hermes отличается от Claude Code, Cursor и OpenClaw

Вы, скорее всего, уже пользуетесь Claude Code, Cursor или OpenClaw. Где здесь место Hermes?

Одной строкой

Claude Code – агент для кодинга внутри репозитория. **Cursor** – напарник по кодированию прямо в редакторе. **OpenClaw** – исполнитель задач, который настраивается через конфиг-файлы. **Hermes** – самообучающийся автономный агент, который живет снаружи всего этого. Большинство продвинутых пользователей используют сразу парочку из них, а не один.

Сравнение

Параметр	Claude Code	Cursor	OpenClaw	Hermes Agent
Основной интерфейс	CLI в репо	IDE	CLI + конфиги	CLI + чат + cron + Telegram
Постоянная память	В рамках сессии	В рамках проекта	Через конфиг-файлы	Между сессиями, с лимитами (MEMORY.md, USER.md)
Обучение	Нет	Нет	Нет	Сам генерирует скиллы из повторяющихся паттернов
Каналы	Терминал	Редактор	Терминал	Терминал, Telegram, Discord, email, вебхуки
Задачи по расписанию	Нет	Нет	Нет	Встроенный cron
Самообучение	Нет	Нет	Нет	Да, через скиллы и память
Привязка к модели	Anthropic	Несколько	Несколько	18+ провайдеров, переключение одной командой
Для чего лучше	Кодинг в репо	Код в редакторе	Shell-процессы	Долгие автономные задачи, эффект которых накапливается

Когда что выбирать

- **Claude Code** – когда вы внутри репо и хотите, чтобы агент читал код, правил его, гонял тесты, коммитил. Это специалист по кодированию.
- **Cursor** – когда нужны AI-автодополнения и правки прямо в редакторе в реальном времени.
- **OpenClaw** – когда хотите описать в конфиге набор shell-команд и запускать их пачкой. Каждая команда – отдельный блок, их можно складывать в цепочки.

- **Hermes** – когда нужен агент, который (а) живет дольше одной сессии, (б) доступен вам через любой удобный канал, (в) со временем становится лучше на ваших повторяющихся задачах.

Скорее всего, вы захотите запустить два

Большинство комьюнити запускает Hermes *вместе* с Claude Code. Claude Code – для кодинга внутри репо. Hermes – для всего остального: ежедневные брифинги, мониторинг, автоматизация, исследования по нескольким репо, координация между каналами. Они не конкурируют, а складываются.

Если мигрируете с OpenClaw, Nous предоставляет утилиту миграции, которая сама портирует конфиги. Порог входа почти нулевой.

4. Установка за 2 минуты

Установка Hermes – одна `curl`-команда плюс два маленьких шага настройки. Если все работает, управитесь меньше чем за две минуты. Если сломается – почти всегда проблема в одной из трех вещей (все разобраны ниже).

Что нужно

- **macOS, Linux или Windows (через WSL2).** Голый Windows официально пока не поддерживается, используйте WSL2 (ссылка на установку на странице Hermes в GitHub).
- **Терминал.** Любой shell подойдет – `bash`, `zsh`, `fish`.
- **API-ключ или локальная модель.** Если у вас еще нет провайдера модели или хочется настроить новый специально под агента – начните с Nous Portal. Или просто возьмите API-ключ у своего провайдера либо установите локальный Ollama/LM Studio. Провайдера модели всегда можно сменить.
- **~200 МБ на диске** под установку плюс сколько понадобится для памяти и скиллов.

Установка

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | sh
```

Это положит бинарь `hermes` в `~/.hermes/bin/` и добавит его в PATH. Установщик печатает все свои действия – никаких скрытых шагов.

Первый запуск

```
hermes
```

При первом запуске Hermes проведет вас через настройку провайдера в интерактивном режиме. Вставьте API-ключ, когда попросит, или укажите локальную модель. Проверьте командой `hermes model`, что настроилось.

Проверка

```
hermes doctor
```

Выводит отчет о здоровье. Все зеленое – можно работать. Красные строки обычно относятся к одной из типовых ошибок ниже.

Три частые ошибки

1. **«Command not found: hermes»** – ваш shell не подхватил новый PATH. Откройте новый терминал или выполните `source ~/.bashrc` (либо `~/.zshrc`).
2. **«Provider error: invalid key»** – в ключе, который вы вставили, есть лишний пробел в конце, или он от другого провайдера. Запустите `hermes model` заново и вставьте аккуратно.

3. **Windows: «bad interpreter»** – вы на голом Windows, а нужен WSL2. Поставьте WSL2 (`wsl --install` в PowerShell), потом перезапустите curl-установщик изнутри WSL.

Все остальное – редкие случаи. Если `hermes doctor` показывает проблему, которую не объясняет, откройте issue в репозитории Hermes – команда Nous быстро разбирает.

5. Выбор модели (главная ошибка первых выходных)

Именно здесь большинство первых настроек тихо ломается. Если Hermes кажется сломанным, тупым или медленным в первой сессии – почти всегда дело в модели, а не в Hermes. Потратьте две минуты здесь – и сэкономите выходные.

Ошибка первых выходных

Hermes сильно делегирует модели. Вызов инструментов, планирование, написание скиллов – это все работа модели. Маленькие или старые модели не могут надежно вызывать инструменты, поэтому застревают в циклах или выдают мусор. **Gemma 2B, Llama 3.1 8B и другие не-frontier модели почти всегда – неправильный выбор по умолчанию.** Они нормально работают для чата, но не для агентских процессов.

Консенсус комьюнити: для реальной работы – frontier-модель, локальная – только для экспериментов.

Цена и возможности

Модель	Ценовой уровень	Вызов инструментов	Скорость	Для чего лучше
Claude Sonnet / Opus	\$\$\$	Отлично	Быстро	Продакшн агентских процессов
GPT-5	\$\$\$	Отлично	Быстро	Продакшн, пользователи OpenAI
z.AI GLM-5	\$	Отлично	Быстро	Лучшее соотношение цена/качество среди frontier
MiniMax M2.7	\$\$	Отлично	Быстро	Отличное соотношение цена/качество
DeepSeek	\$	Хорошо	Быстро	Экономичные процессы
Nous Portal (подписка)	\$ (фикс)	Отлично	Быстро	Предсказуемая месячная стоимость
Ollama + Qwen 2.5 Coder	Бесплатно (локально)	Хорошо	Зависит от GPU	Офлайн-кодинг, работа с приватными данными
Ollama + Gemma / Llama 8B	Бесплатно (локально)	Плохо	Зависит от GPU	Только чат; не для агентской работы

Когда локальная модель подходит, а когда нет

- **Локальная подходит, когда:** вы пишете код фрагментами, общаетесь или делаете одноразовые саммаризации. Ollama + Qwen 2.5 Coder 32B покрывает 80% повседневного использования в CLI.
- **Локальная не подходит, когда:** нужны многошаговые вызовы инструментов (следить за сайтом, вызвать API, суммаризировать, отправить в Telegram). Будут

циклы, таймауты, кривые аргументы у инструментов. Переключайтесь на frontier-API.

Кнопка «просто работай»: Nous Portal

Если не хочется возиться с API-ключами разных провайдеров, Nous Portal – это одна подписка, которая покрывает Claude, GPT, GLM, MiniMax, собственные модели Nous и многое другое – через одну авторизацию. Входите через OAuth из CLI. Платите одну фиксированную сумму в месяц и перестаете думать, какой ключ куда. Большинство в итоге оказывается здесь.

6. Ваш первый рабочий процесс

Не пытайтесь собрать три вещи одновременно. Выберите один из примеров ниже и запустите его сегодня. Каждый займет 10–30 минут и научит вас разной половине Hermes.

Пример 1 – ассистент по коду на вашем ноутбуке

```
cd ~/projects/my-repo
hermes
> проведи аудит этого репо на мертвый код,
   неиспользуемые импорты и закомментированные блоки
   старше 6 месяцев. Выдай markdown-отчет.
```

Что вы заметите: Hermes читает файлы, запускает grep-ы и пишет отчет в `./hermes-output/` по умолчанию. Во второй раз такая же задача выполнится быстрее – потому что Hermes на первом запуске написал скилл `repo-audit`.

Почему это хороший первый процесс: нулевая инфраструктура, осязаемый результат и система скиллов покажет себя за один вечер.

Пример 2 – Telegram-бот на VPS за \$5

На любом дешевом Linux VPS (Hetzner, Digital Ocean, Hostinger, Raspberry Pi – не важно):

```
# установить Hermes
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | sh

# настроить Telegram-шлюз
hermes gateway setup
# вставить токен бота от @BotFather
# вставить свой числовой Telegram user ID
# (использовать @userinfobot) - HE @username

# запустить демон
hermes daemon start
```

Напишите боту – он ответит. Теперь добавьте задачу:

```
> каждое утро в 8:00 суммаризируй мои уведомления
   с GitHub и присылай их мне сюда
```

Hermes запишет cron-задачу, сохранит ваши предпочтения в память и пингнет вас завтра в 8:00. Оставьте VPS работать.

Типичная ловушка: в allowlist Telegram нужен именно числовой user ID (число вроде 123456789), а не @handle. Если указать handle, бот тихо примет сообщения от любого, кто угадает ваш хэндл. Свой числовой ID можно получить у @userinfobot в Telegram.

Почему это хороший первый процесс: вы уходите с работающим всегда-включенным агентом за \$5/месяц. Все остальные процессы будут переиспользовать эту схему.

Пример 3 – утренний брифинг по cron

Ни бота, ни чата. Просто запланированная задача, которая каждое утро присылает одну сводку на email или в мессенджер.

```
hermes
> каждый будний день в 7:30 собирай:
  (1) топ-10 постов Hacker News за ночь,
  (2) мои непрочитанные уведомления с GitHub,
  (3) погоду в Сиэтле.
Отправляй как обычный email на me@example.com.
```

Hermes планирует cron-задачу, запоминает email и город и начинает ее выполнять. Проснитесь завтра – проверьте инбокс.

Почему это хороший первый процесс: низкая поверхность, всегда включено, и вы увидите, как Hermes относится к запланированным процессам как к полноценным гражданам.

7. Цикл обучения

Этот раздел стоит прочитать внимательно. Если вы поняли цикл обучения – вы поняли, почему Hermes устроен именно так.

Почему Hermes со временем становится лучше

После каждой задачи – или каждых примерно пяти вызовов инструментов – Hermes проводит с собой короткий разбор: *сработало ли? что заняло слишком долго? что отклонил пользователь? есть ли здесь переиспользуемый паттерн?* Когда ответ – да, он пишет новый скилл или обновляет память. В следующий раз при похожем запросе Hermes сначала тянется к скиллу и пропускает переоткрытие.

Это противоположность чата без состояния. Чат без состояния выбрасывает все в конце разговора. Hermes записывает.

Ограниченная память – это фича, а не баг

Память хранится в трех местах:

- **MEMORY.md** – краткосрочная, максимум ~2 200 символов. То, над чем агент работает прямо сейчас.
- **USER.md** – факты о вас, максимум ~1 375 символов. Имя, предпочтения, роль, ограничения.
- **Поиск по сессиям** (полнотекстовый индекс FTS5) – каждая прошлая беседа доступна для поиска.

Маленькие лимиты – осознанное решение. Безлимитная память звучит заманчиво, пока вы не осознаете: так агенты превращаются в помойку устаревшего контекста. Hermes заставляет консолидировать: когда **MEMORY.md** заполняется, агент должен решить, что стоит сохранить. Эта дисциплина держит рабочий контекст компактным, а модель – сфокусированной.

Если хочется безлимитной семантической памяти, есть восемь комьюнити-провайдеров памяти (Honcho, Mem0, Hindsight, Supermemory и другие), которые подключаются поверх.

Скиллы – это выполняемые процедуры, а не заметки

Скиллы – часть, которую все недооценивают. Скилл – это markdown-файл по адресу `~/.hermes/skills/<name>/SKILL.md`, в котором описано:

- **Когда** триггерить (короткое описание, которое читает оркестратор)
- **Как** выполнять (инструкции, вызовы инструментов, пример входа/выхода)
- **Контекст** нужен (какие файлы, какие API)

Память Claude Code хранит *факты* о вас («предпочитает буллеты»). Hermes хранит *процедуры* («процесс ресерч – фильтр – формат, который выдает буллеты так, как вам нужно»). Это существенная разница. Факты информируют модель. Процедуры переиспользуются напрямую.

Правило пяти вызовов инструментов

Скиллы не генерируются после каждой задачи – так было бы шумно. Они появляются, когда Hermes видит повторение: обычно после пяти и более вызовов инструментов на один паттерн или после пользовательской правки, которая учит чему-то обобщенному. Поэтому в первый день ваша папка со скиллами пуста, а к тридцатому дню там 20+ скиллов, которые рулят вашей жизнью.

Что на самом деле значит «Harness Engineering»

Nous часто использует словосочетание *Harness Engineering* – инженерия обвязки. Это рамка всего продукта: LLM – заменимый компонент, а настоящая инженерная работа идет в слоях вокруг. Их пять: слой инструкций (как вы говорите с моделью), слой ограничений (что ей разрешено), слой обратной связи (как ошибки возвращаются), слой памяти (что сохраняется), слой оркестрации (как несколько инструментов и вызовов сшиваются вместе). Hermes – мнение команды о том, как реализовать эти пять слоев. Меняйте модель – обвязка остается.

8. Обязательные скиллы для установки в первый день

В Hub-е 643 комьюнити-скилла (на 19.04.2026). Не пытайтесь поставить все. Вот пять, которые стоит иметь каждому пользователю Hermes с первого дня.

Что такое Hub на 643 скилла

Запустите `hermes skills browse` – увидите каталог. Скиллы приходят из четырех уровней доверия:

- **Builtin** – идут в комплекте с Hermes, изменить нельзя.
- **Official** – публикуются Nous, с аудитом безопасности.
- **Trusted** – публикуются верифицированными участниками комьюнити (OpenAI, Anthropic, VoltAgent, gstack и т.д.).
- **Community** – все остальное; сканируется на проблемы с безопасностью, но перед установкой стоит посмотреть исходники.

Пять, которые стоит поставить сегодня

1. **LLM Wiki** (builtin) – компактный справочник по любой теме в духе Карпатого. Даете «трансформеры» – получаете двухминутный праймер. Хорошо для вкатывания во все, с чем сталкивается Hermes.
2. **gstack** (trusted) – автоматизация браузера, QA-тестирование и дизайн-ревью в одном скилле. Обязательно для тех, кто использует Hermes для проверки сайтов.
3. **OpenAI taps** (trusted) – подтягивает примитивы function-calling от OpenAI как инструменты Hermes. Полезно, даже если вы не на модели OpenAI.
4. **Manim** (official) – генерация математических анимаций из текста. Неожиданно затягивает всех, кто пишет технический контент.
5. **security-audit** (official) – сканирует репо на распространенные уязвимости. Запускайте перед тем, как публиковать что-то наружу.

Установка: `hermes skills install <name>`

Команды для работы с Hub

Чтобы искать и смотреть скиллы, используйте:

```
hermes skills browse          # интерактивный браузер
hermes skills search telegram # поиск по ключевым словам
hermes skills info <name>     # посмотреть до установки
```

Каждый скилл показывает уровень доверия, автора, описание и последние изменения. Для уровня Community обязательно смотрите это до запуска.

9. Куда двигаться дальше

Вы поставили Hermes, выбрали модель, запустили один процесс и поняли цикл обучения. Дальше – три пути.

Путь 1: глубже в агентов

Соберите команду из нескольких агентов. Один оркестратор, один ресерчер, один писатель, один отладчик – у каждого своя память и свои скиллы. Система делегирования Hermes делает это на удивление чисто.

Путь 2: глубже в инструментарий

Собирайте собственные инструменты и скиллы. Изучите протокол MCP, чтобы ваш Hermes общался с Notion, Linear, Figma, внутренними API. Здесь ментальная модель Harness Engineering раскрывается в полной мере.

Путь 3: глубже в инфраструктуру

Запустите Hermes в продакшне для команды. Docker, systemd, готовые облачные шаблоны, Kubernetes. Комьюнити уже упаковало большую часть этого.

10. FAQ

Нермес Агента бесплатен?

Сам Нермес – бесплатный и опенсорсный (лицензия MIT). Вы платите только за API модели, которую используете. С некоторыми API-провайдерами или локальной Ollama общая стоимость может быть \$0. С Claude или GPT легко набежит \$100+ в месяц в зависимости от объема.

Работает ли на Windows?

Да, через WSL2. Голый Windows официально не поддерживается. Запустите `wsl --install` в PowerShell, потом ставьте Нермес изнутри WSL как на Linux. Комьюнити спокойно гоняет Нермес на Windows таким способом.

Можно ли запустить полностью офлайн?

Да. Поставьте Нермес, свяжите с Ollama или LM Studio и отключите сеть. Память, скиллы и стоп работают офлайн. Ловушка: локальные модели слабее в вызове инструментов, поэтому сложные многошаговые процессы проседают. Для офлайн-кодинга – в рамках возможностей вашей локальной модели нормально. Для офлайн-автоматизации с множеством вызовов инструментов ждите шероховатостей.

Нужен ли GPU?

Нет – если используете API-провайдера, любой ноутбук потянет Нермес. GPU нужен только для локальной модели.

Мои данные куда-то отправляются?

Только тому провайдеру, которого вы настроили. У Нермес нет телеметрии, нет аналитического маячка, нет «звонка домой». Память, скиллы и логи сессий живут на вашем диске. Если выбрана локальная модель – данные вообще не покидают вашу машину.

Нермес vs Claude Code – нужно ли выбирать один?

Нет. Большинство продвинутых пользователей запускают оба. Claude Code – для кодинга внутри репо. Нермес – для всего вне репо (автоматизация, мониторинг, чат, cron, исследования между репо). Они дополняют друг друга, а не конкурируют.

Может ли Нермес сломать мой код?

Нет – если вы сами не нацелите его на свою кодовую базу и не попросите. Нермес аддитивен: память, скиллы и логи живут в `~/.hermes/`. Файлы вне директорий, с которыми вы ему позволили работать, он не трогает. Хотите более строгую песочницу – включите Docker-бэкенд и политики подтверждений.

Что, если он что-то сломает?

У Hermes есть политики подтверждений, которые требуют подтверждения на деструктивные операции (удаления, `force-push`, `rm -rf`). Включите их – по умолчанию они выключены, чтобы новых пользователей не заваливало промптами. Если что-то пойдет не так, каждая сессия пишется в `~/hermes/sessions/` – большинство операций обратимо.

Может ли команда делить одного агента?

Да, через профили. Каждый профиль – изолированный инстанс агента со своей памятью, сессиями, скиллами и stop-состоянием. Создайте общий профиль на общем VPS, выдайте команде доступ через allowlist мессенджер-шлюза – и у вас командный агент.

Можно ли использовать с Ollama или LM Studio?

Да. Запустите `hermes model`, выберите «custom OpenAI-compatible endpoint», укажите ваш локальный сервер (обычно `http://localhost:11434/v1` для Ollama). Все работает идентично API-провайдерам, кроме качества вызова инструментов – оно зависит от локальной модели.

Как обновить?

```
hermes update
```

Hermes обновляет сам себя. Показывает, что изменилось, бэкапит конфиг, применяет новую версию. Текущая версия – v0.10.0 (на 20.04.2026), команда релизит примерно раз в две недели.

Источник: hermesatlas.com/guide. Автор – Kevin Simback, мейнтейнер Hermes Atlas.